

RPG Manager

Sistema de Gerenciamento de Jogo de RPG

Descrição de Projeto de Disciplina

Programação Orientada a Objetos com Python e C++

UTFPR — Universidade Tecnológica Federal do Paraná

1. Identificação do Projeto

Nome do Projeto	RPG Manager — Sistema de Gerenciamento de Jogo de RPG
Disciplina	Programação Orientada a Objetos
Linguagem	C++17
Carga Horária	40 horas (projeto semestral)
Modalidade	Individual ou dupla
Nível	Graduação — Engenharia da Computação

2. Introdução e Motivação

Jogos de RPG (*Role-Playing Game*) são sistemas complexos que envolvem múltiplas entidades com atributos, comportamentos e relacionamentos bem definidos, tornando-os um domínio extremamente rico para a prática de Programação Orientada a Objetos (POO). A modelagem de personagens, habilidades, inventários e regras de combate é um exercício natural de abstração, encapsulamento, herança e polimorfismo.

Este projeto propõe o desenvolvimento do **RPG Manager**, um sistema de gerenciamento de jogo de RPG implementado em C++. O objetivo central é aplicar os principais conceitos de POO para construir um sistema funcional, extensível e bem estruturado.

3. Objetivos

3.1 Objetivo Geral

Desenvolver um sistema de gerenciamento de jogo de RPG utilizando os paradigmas e conceitos de Programação Orientada a Objetos em C++, consolidando o aprendizado prático das linguagens e das boas práticas de design de software.

3.2 Objetivos Específicos

- Modelar hierarquias de classes para personagens, itens e habilidades usando herança e polimorfismo
- Implementar encapsulamento e controle de acesso a atributos de entidades do jogo
- Utilizar sobrecarga de operadores e métodos especiais em C++
- Aplicar composição e agregação para relacionamentos entre entidades (personagem e inventário)
- Desenvolver um sistema de combate por turnos utilizando padrões de projeto básicos
- Produzir código documentado, testado e organizado em módulos/namespaces

4. Descrição do Sistema

O RPG Manager é um sistema de linha de comando (CLI) que simula o gerenciamento de um jogo de RPG clássico. O sistema deve ser capaz de criar e gerenciar personagens, controlar inventários, executar batalhas por turnos e persistir o estado do jogo.

4.1 Módulos do Sistema

Módulo	Descrição	Conceitos POO
Personagens	Hierarquia de classes: Herói, Mago, Guerreiro, Arqueiro, etc.	Herança, Polimorfismo
Inventário	Gerenciamento de itens, slots, peso, equipamentos e consumíveis	Composição, Encapsulamento
Habilidades	Sistema de skills ativas e passivas com efeitos variáveis	Polimorfismo, Abstração
Combate	Sistema de batalha por turnos com logs de ações	Padrão Strategy/Observer
Persistência	Serialização e carregamento do estado do jogo (JSON / binário)	Sobrecarga de operadores

4.2 Hierarquia de Classes Principal

A estrutura de classes central do sistema pode seguir a seguinte organização mínima:

- **Classe abstrata Entidade:** base para todos os seres do jogo (atributos comuns: nome, pontos de vida, nível)

- **Classe Personagem** (herda de **Entidade**): adiciona classe RPG, raça RPG, experiência, inventário e habilidades
- **Classes concretas: Guerreiro, Mago, Arqueiro, Clérigo** — cada uma com atributos e métodos especializados
- **Raças concretas: Humano, Elfo, Anão, Orc** — cada uma com atributos e métodos especializados
- **Classe Item**: base para todos os itens, com subclasses **Arma, Armadura, Poção** e **ItemEspecial**
- **Classe Inventário**: composição com lista de **Item**, controle de capacidade e peso
- **Classe abstrata Habilidade**: polimorfismo para habilidades ofensivas, defensivas e de suporte
- **Classe Batalha**: gerencia o *loop* de combate por turnos entre dois ou mais personagens

5. Requisitos do Projeto

5.1 Requisitos Funcionais

1. Criação de personagens com escolha de classe, raça, distribuição de atributos e nome
2. Listagem e descrição detalhada de personagens criados
3. Adição, remoção e uso de itens no inventário de um personagem
4. Equipar e desequipar armas e armaduras, com atualização automática dos atributos
5. Execução de batalha por turnos entre dois personagens (humano vs. humano ou humano vs. *bot*)
6. Ganho de experiência e sistema de *level up* com redistribuição de pontos
7. Salvar e carregar o estado do jogo a partir de arquivo
8. Menu interativo em linha de comando (CLI) para todas as operações

5.2 Requisitos Técnicos — C++

- Implementar herança pública, protegida e privada conforme adequado
- Utilizar funções virtuais puras e polimorfismo com ponteiros/referências
- Aplicar sobrecarga de operadores ($\ll$, $\equiv$, $<$, $+$) onde semanticamente válido
- Organizar código em *namespaces* e arquivos `.h/.cpp` separados
- Utilizar STL: `vector`, `map`, `list`, `algorithm` para coleções de entidades
- Escrever testes unitários com Google Test (`gtest`) ou similar

6. Entregáveis e Avaliação

6.1 Entregáveis

1. Código-fonte completo em C++ (diretório `rpg_manager_cpp/`)
2. Diagrama de classes UML em formato digital (draw.io, PlantUML ou equivalente)
3. Relatório técnico em PDF descrevendo as decisões de design
4. Apresentação oral de 15 minutos com demonstração do sistema em execução

6.2 Critérios de Avaliação

Critério	Peso	Pontuação Máx.
Implementação correta dos conceitos de POO (herança, polimorfismo, encapsulamento, abstração)	30%	30 pts
Funcionamento correto do sistema (todos os requisitos funcionais atendidos)	25%	25 pts
Qualidade do código (organização, legibilidade, documentação, boas práticas)	15%	15 pts
Diagrama UML completo e coerente com a implementação	15%	15 pts
Apresentação oral e demonstração	15%	15 pts

6.3 Critérios de Avaliação do Projeto

Critério	Descrição
Funcionalidade	Todas as features implementadas e funcionando corretamente
Design OO	Uso adequado de herança, composição e polimorfismo
Padrões de Projeto	Aplicação correta de pelo menos 4 padrões de projeto
Qualidade do Código	Legibilidade, organização e nomenclatura adequadas
Tratamento de Erros	Exceções bem definidas e tratadas em todos os módulos
Interface Gráfica	Funcional, intuitiva e bem integrada ao sistema
Testes	Cobertura mínima de 60% do código crítico
Documentação	Código documentado, README completo e diagrama UML

7. Recursos Online

- Documentação oficial C++: <https://en.cppreference.com>
- Documentação Qt: <https://doc.qt.io>

- **LearnCpp:** <https://www.learncpp.com>
- **Refactoring Guru (Design Patterns):** <https://refactoring.guru>

8. Normas Gerais

- Todo código deve ser versionado no Git
- Commits frequentes são esperados (mínimo 2–3 por quinzena/mês)
- Código deve ser documentado conforme padrões da linguagem (Doxygen em C++)
- Plágio de código será tratado conforme regulamento acadêmico da UTFPR
- Entregas atrasadas terão desconto de 20% por dia (máximo 3 dias de tolerância)

9. Extensões Opcionais (Bônus)

Os itens a seguir são opcionais e podem ser implementados para pontuação adicional:

- Interface gráfica simples **SFML** ou **QT** (C++)
- Sistema de missões (*quests*) com recompensas e rastreamento de progresso
- Implementação do padrão de projeto *Observer* para eventos de batalha (ex.: notificações ao subir de nível)
- Sistema de *crafting*: combinação de itens para criar equipamentos melhores
- Persistência em banco de dados SQLite em vez de arquivos texto/JSON
- Mapa de exploração com salas e inimigos gerados proceduralmente

10. Referências

GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

STROUSTRUP, B. *The C++ Programming Language*. 4. ed. Addison-Wesley, 2013.

LUTZ, M. *Learning Python*. 5. ed. O'Reilly Media, 2013.

MARTIN, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

FOWLER, M. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*.
3. ed. Addison-Wesley, 2003.